

Arboles y grafos

Notación asintótica

Notación O

Notación Ω

Notación Θ

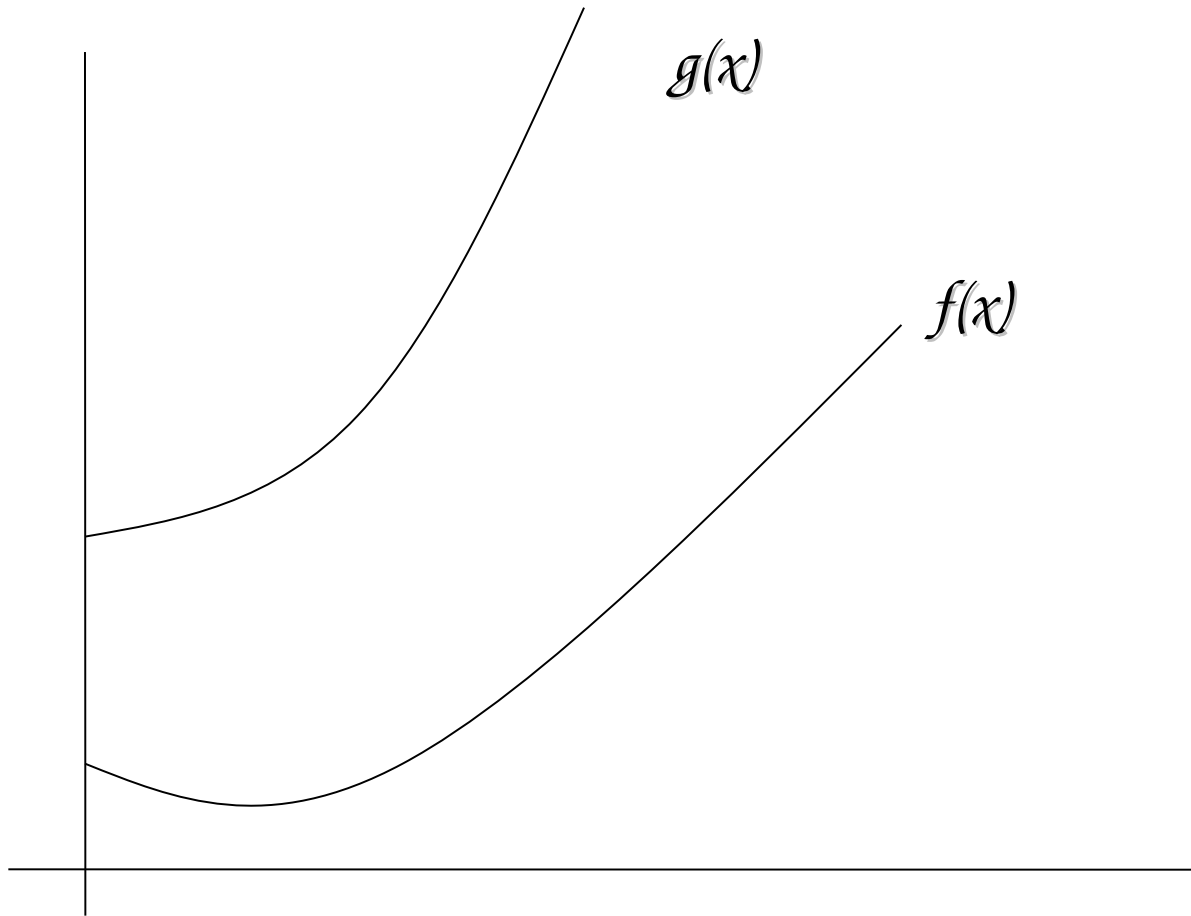
Notación o

Notación ω

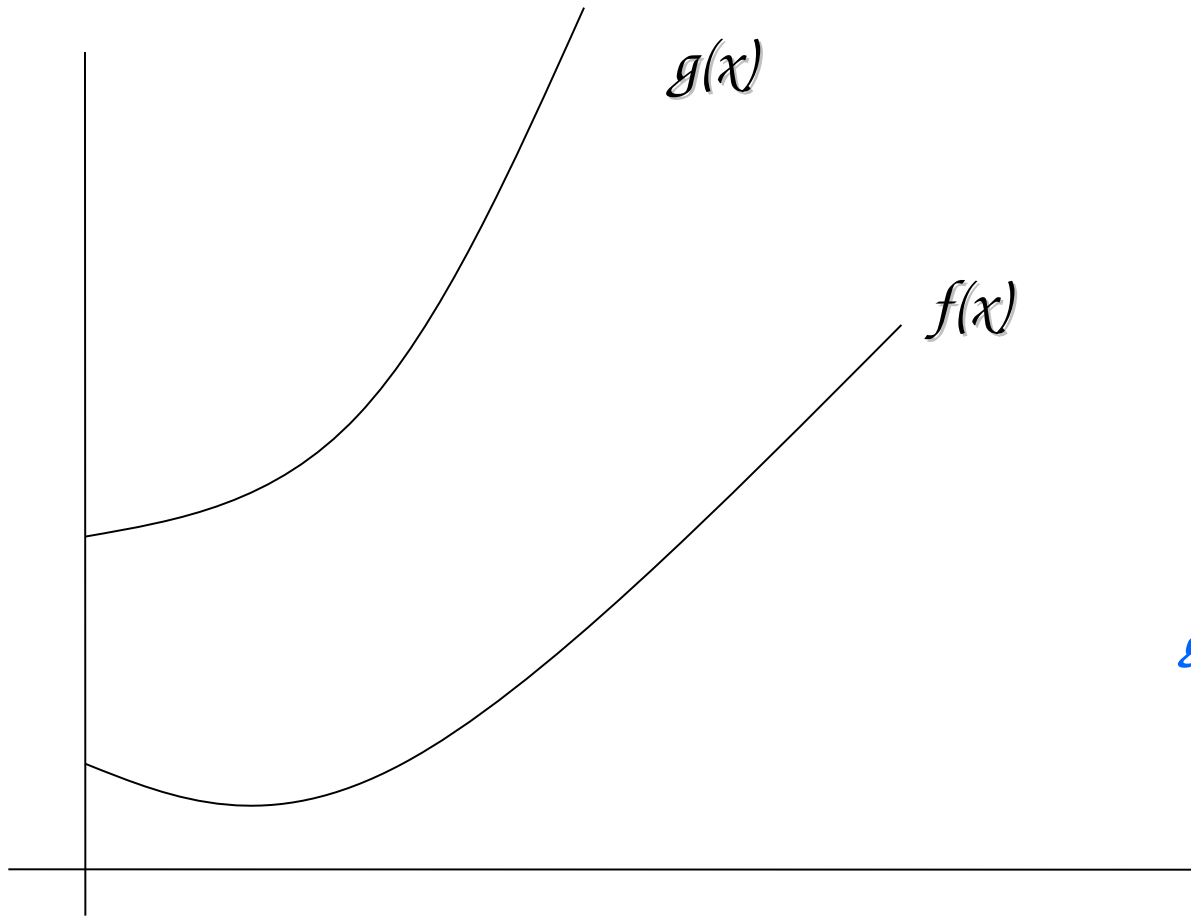
Terminología de complejidades

Clasificación de problemas

Crecimiento de funciones

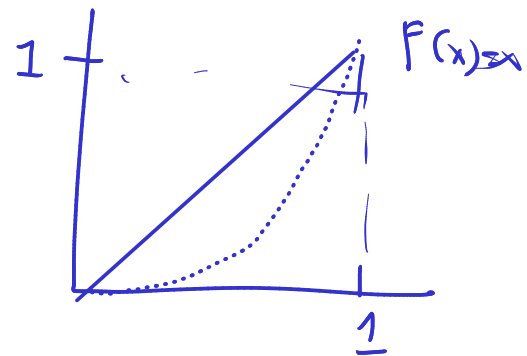
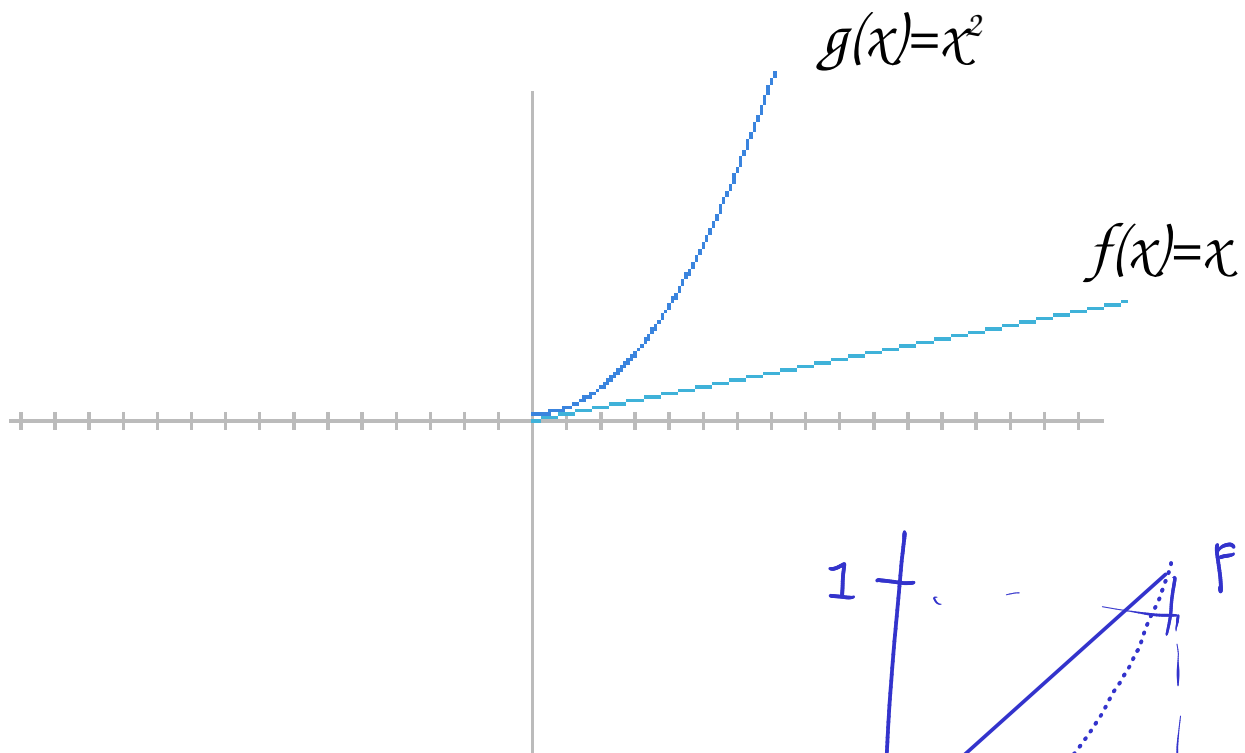


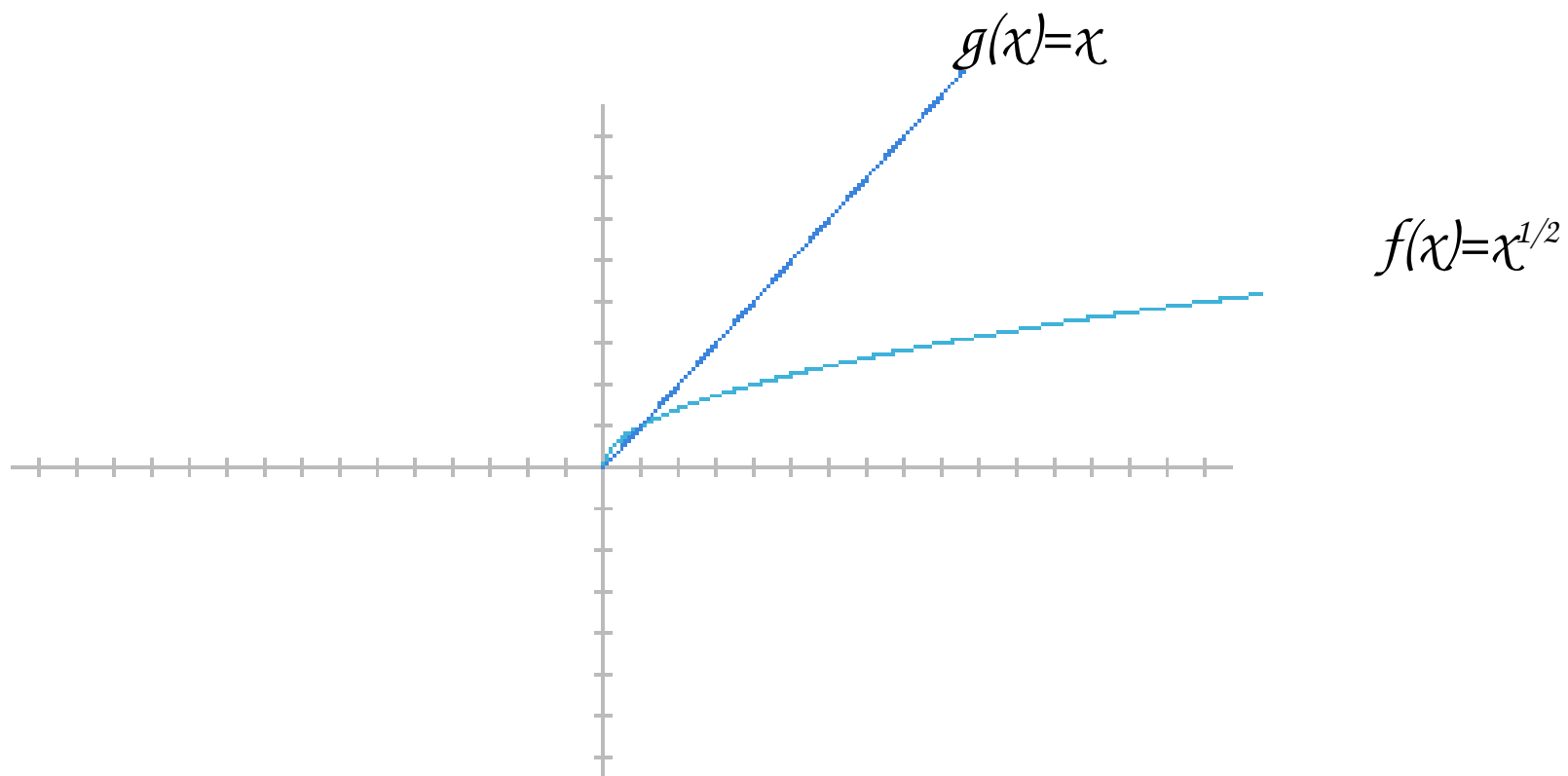
Crecimiento de funciones

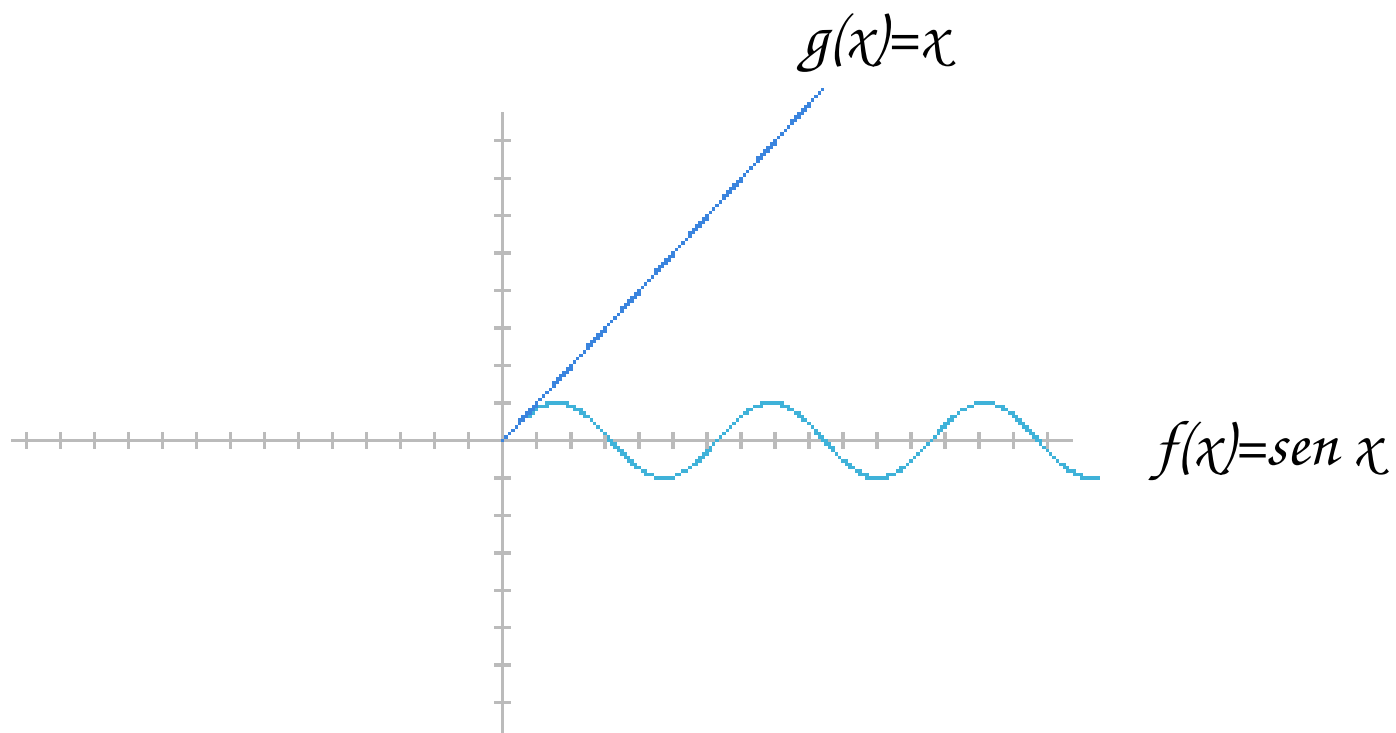


*Siempre se
cumple que
 $f(x) < g(x)$ para
cualquier valor
 x*

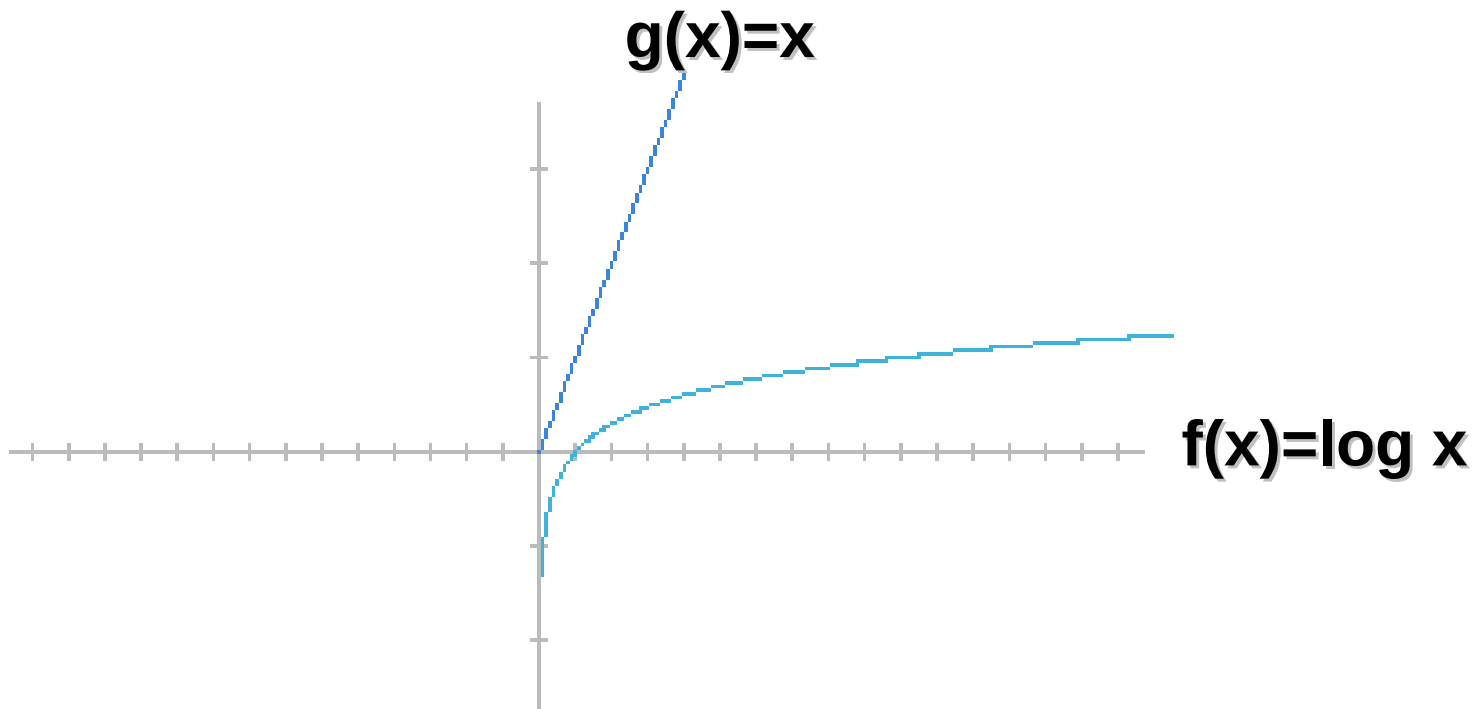
*$g(x)$ es una cota superior
de $f(x)$*

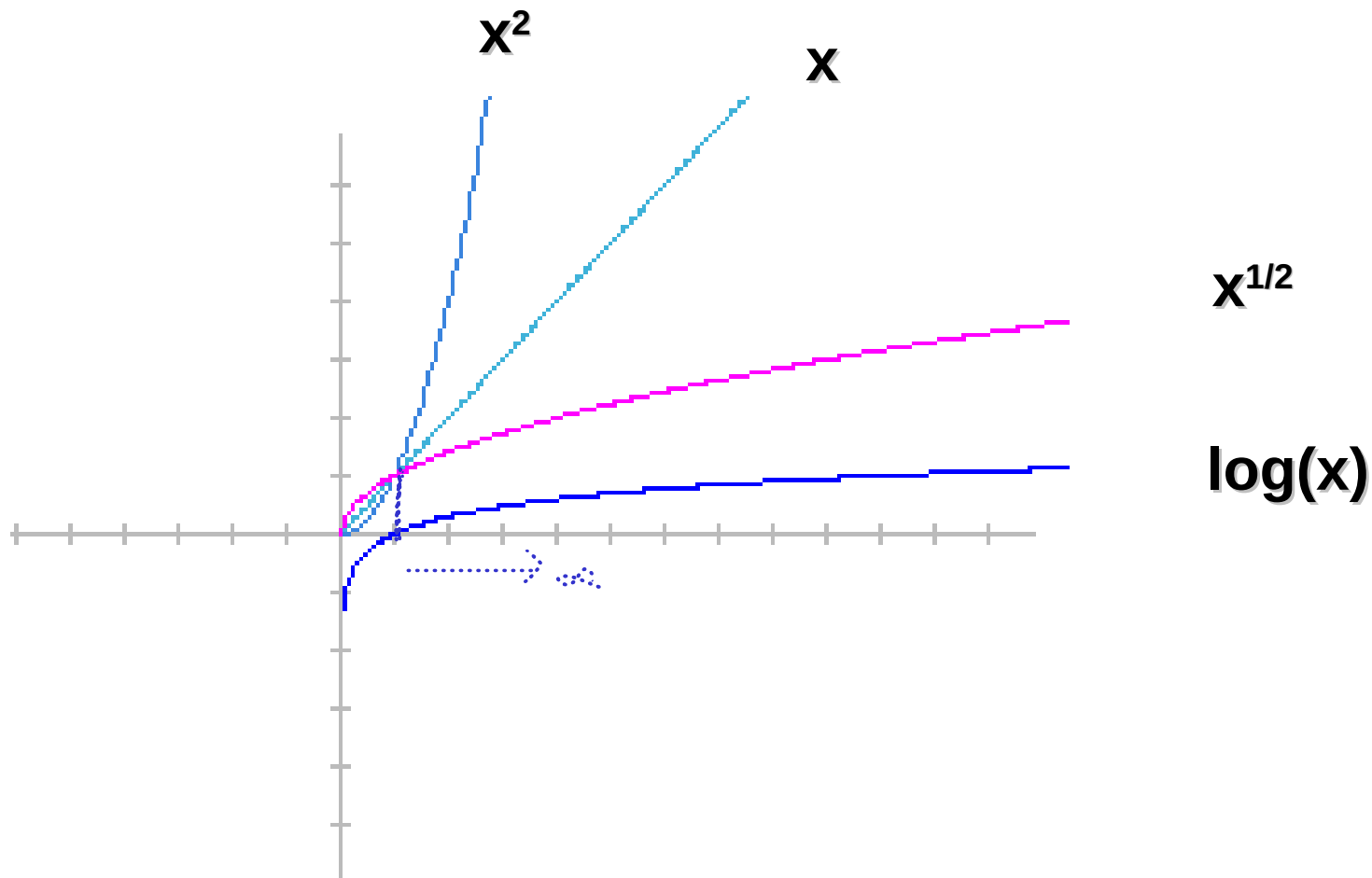






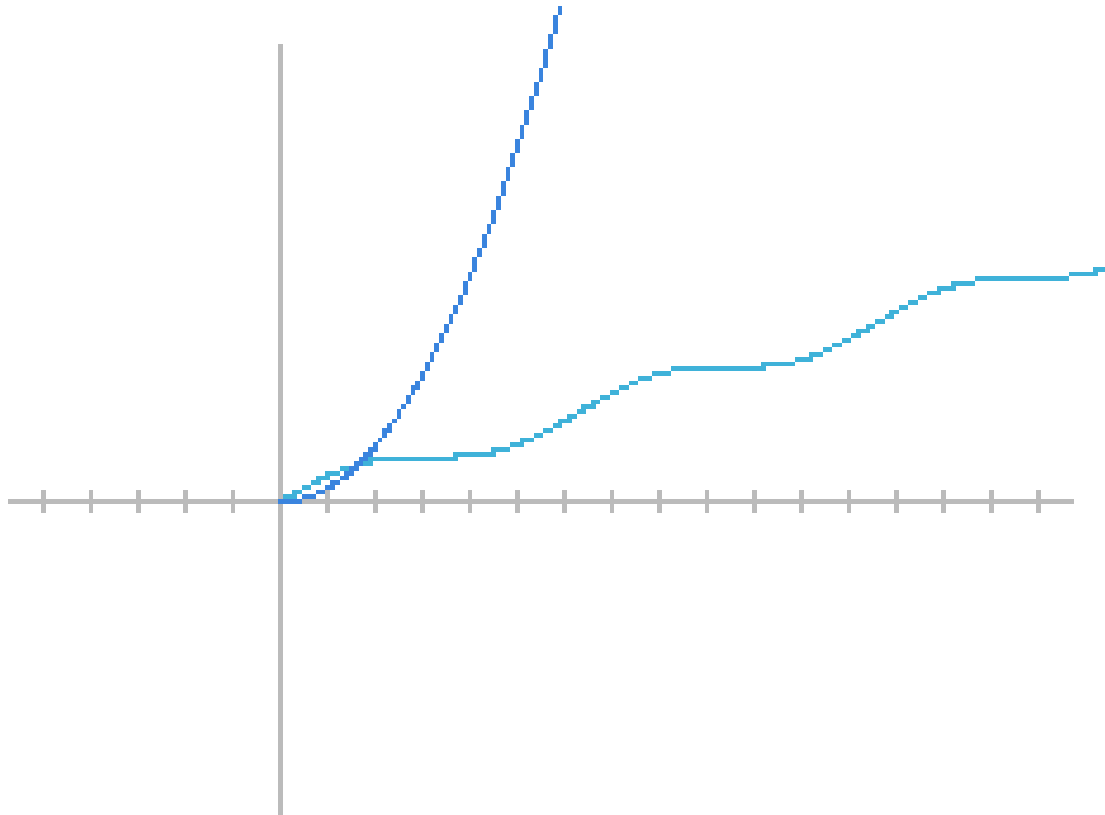
*¿Está ($\text{sen } x$) por
debajo de x^2 ?*





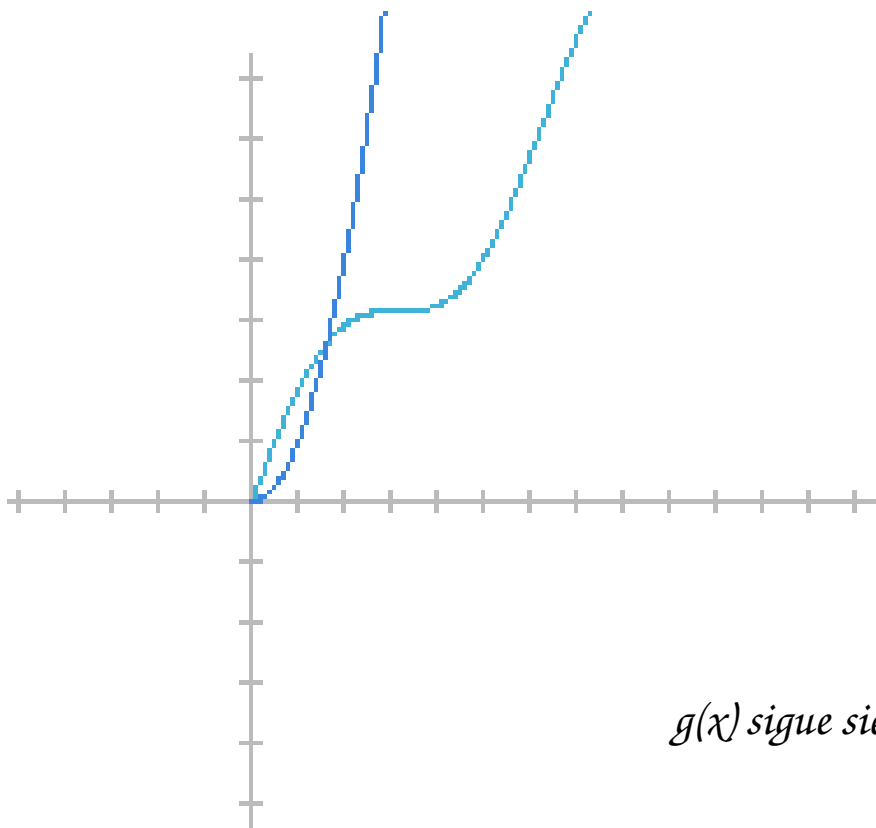
$$g(x)=x^2$$

$$f(x)=\sin(x) + x$$

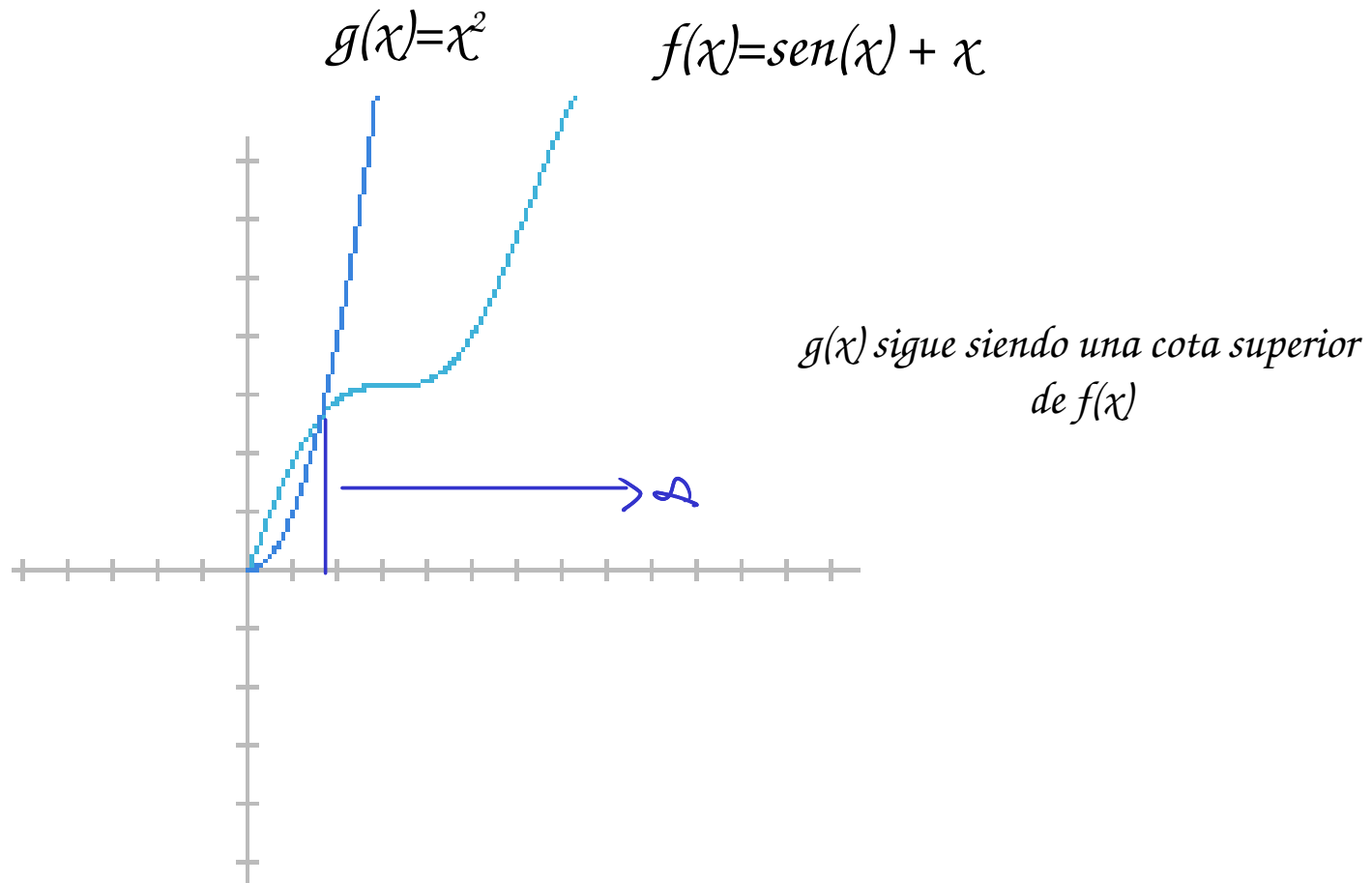


$$g(x)=x^2$$

$$f(x)=\sin(x) + x$$



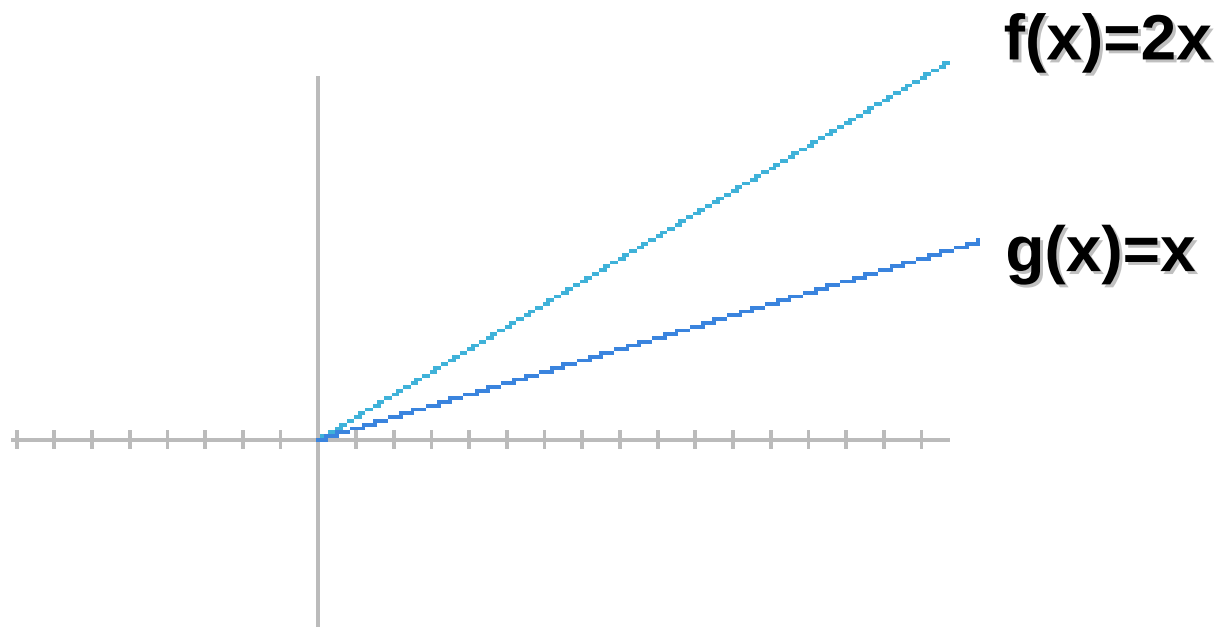
*$g(x)$ sigue siendo una cota superior
de $f(x)$*



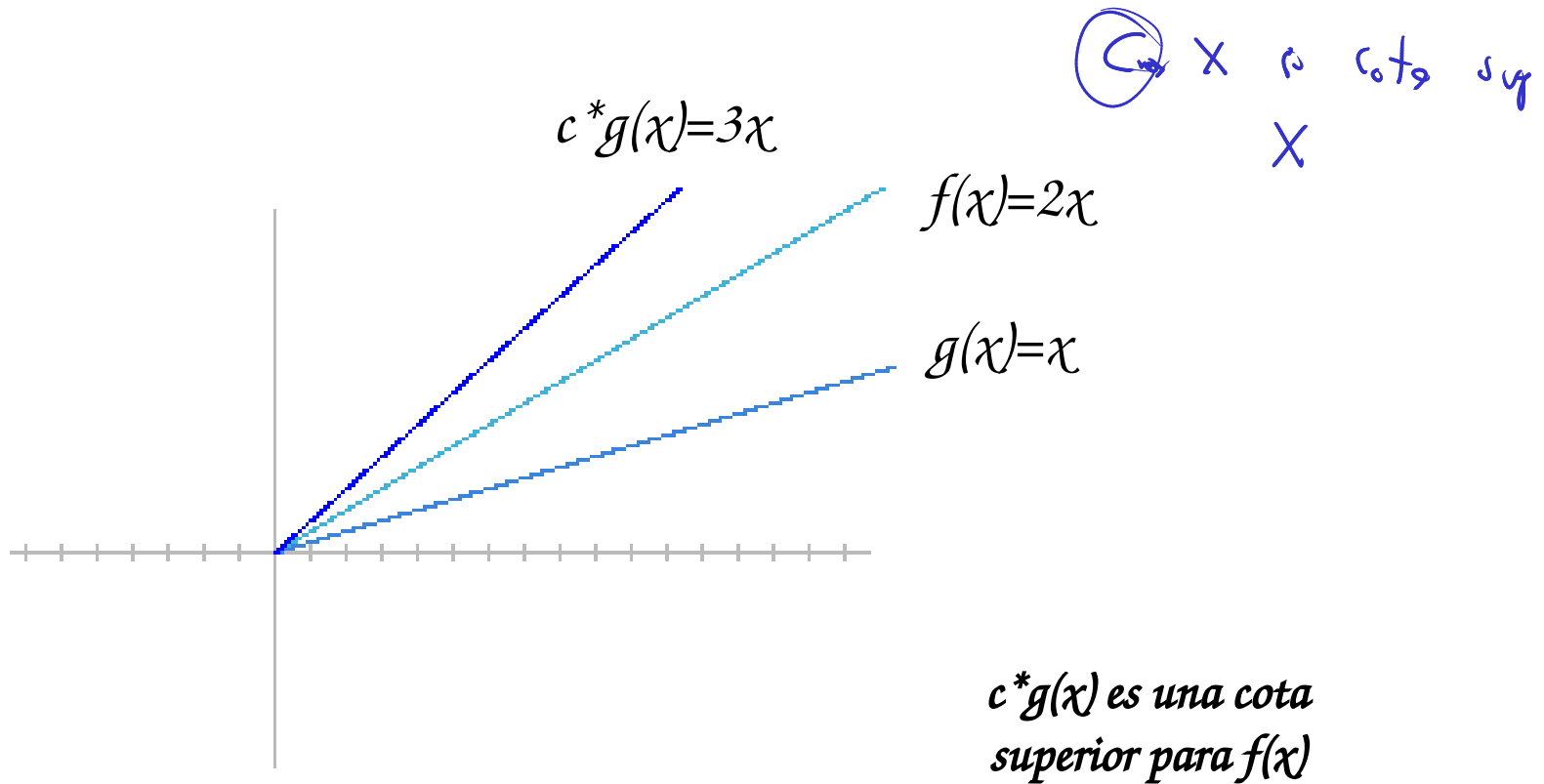
Para que una función sea cota superior de otra, debe serlo para todos los valores de $x \geq k$

Pueden existir valores de $x < k$ para los cuales no sea cota superior

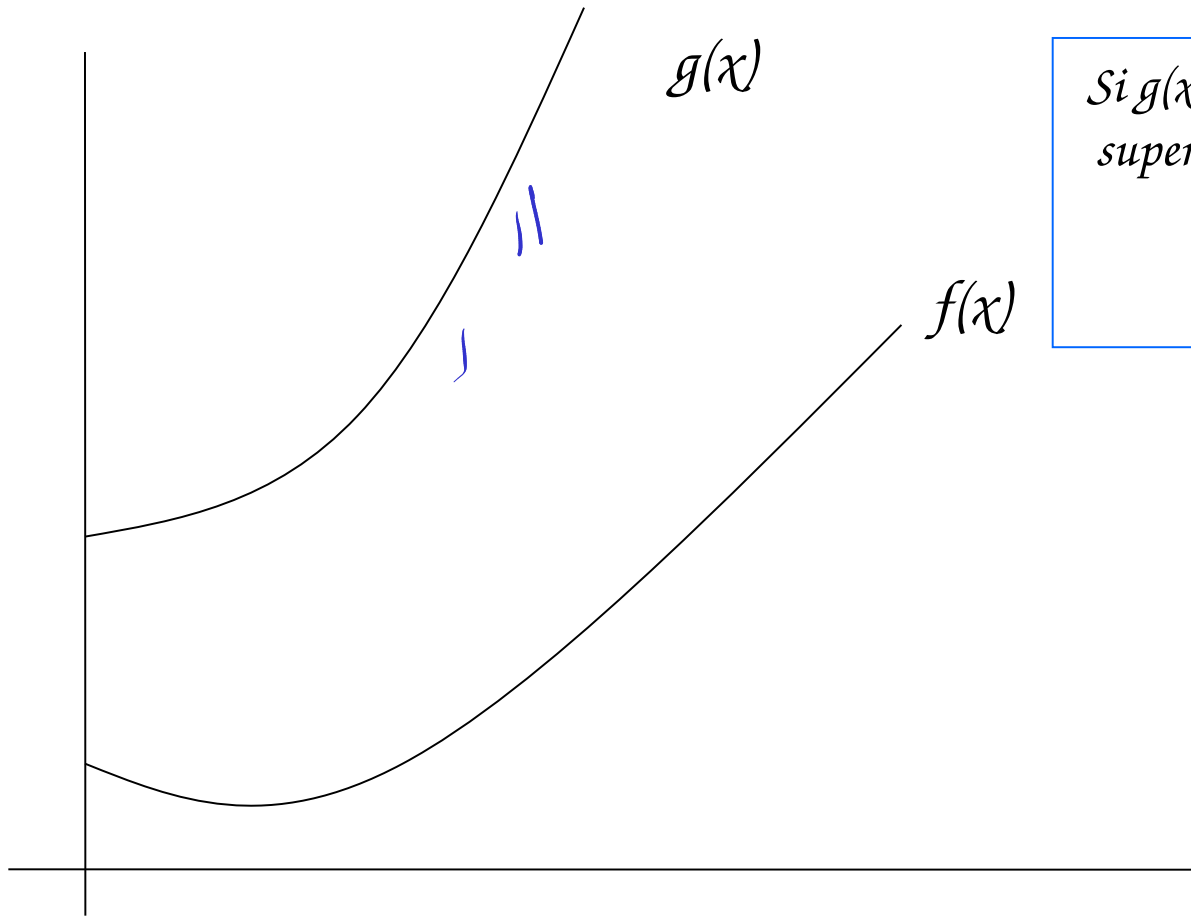
Buscar una cota superior para $f(x)$ en términos de $g(x)$



Crecimiento de funciones



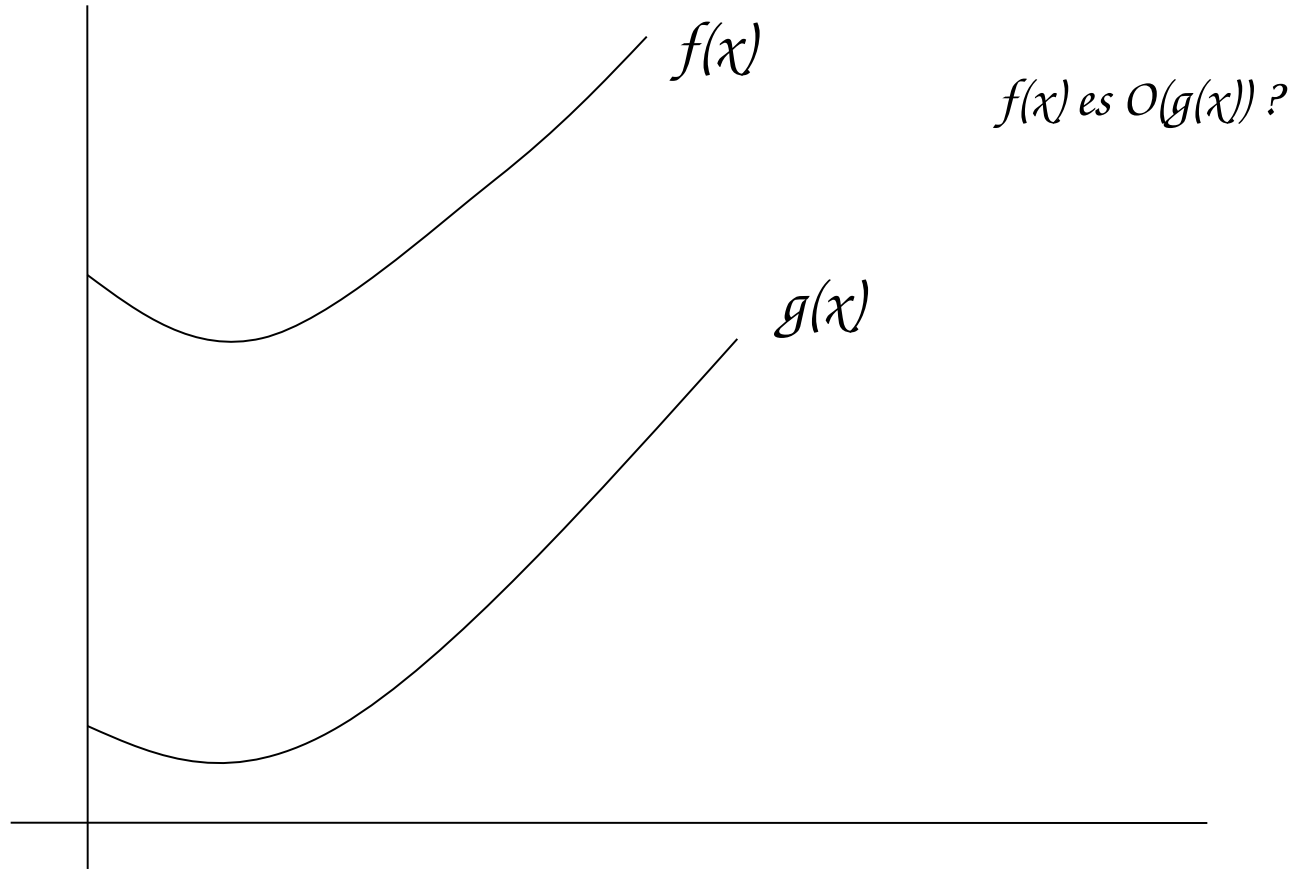
Crecimiento de funciones



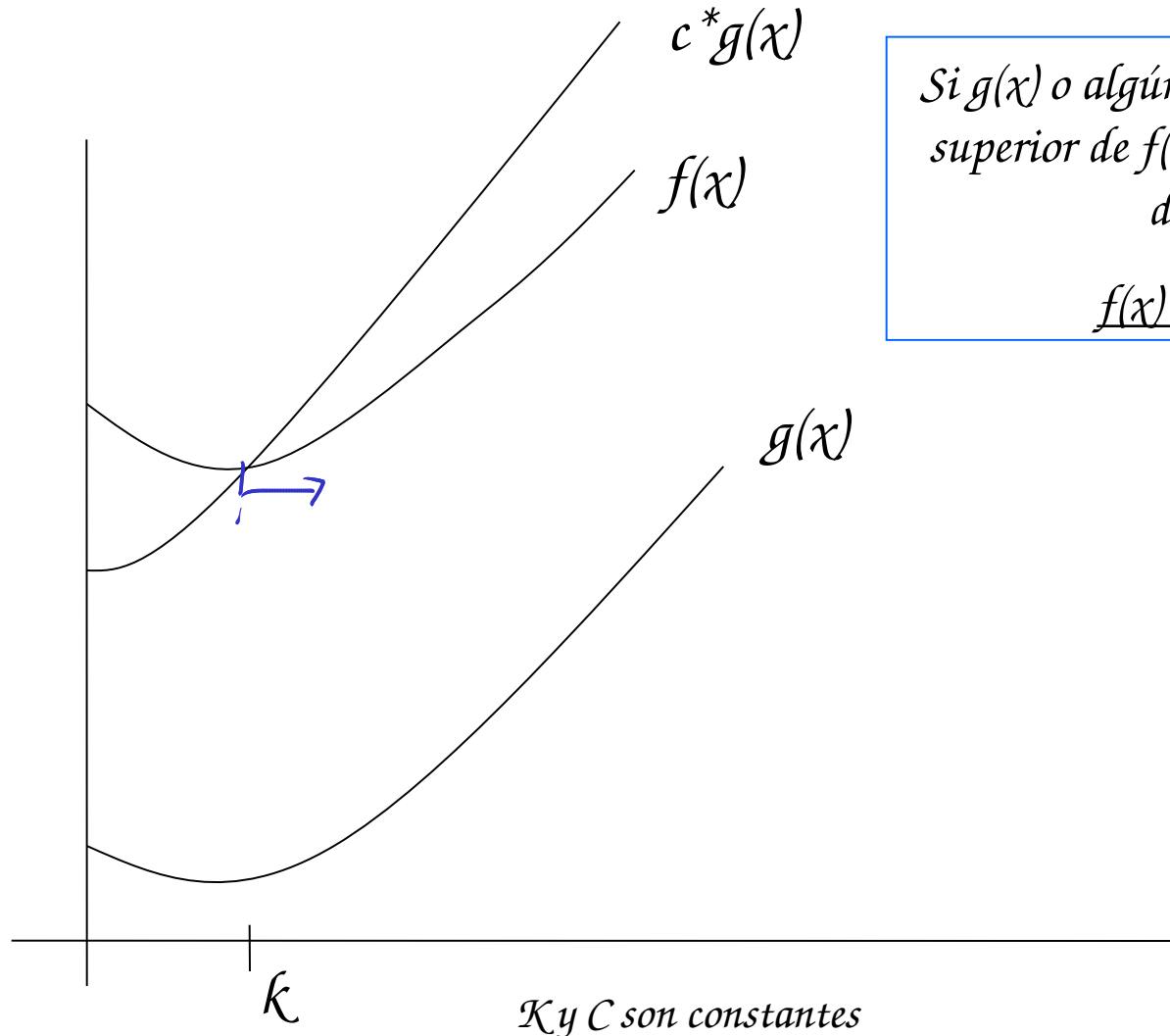
Si $g(x)$ o algún $c^*g(x)$ es una cota superior de $f(x)$, para los $x > k$, se dice que

$f(x)$ es $O(g(x))$

Crecimiento de funciones



Crecimiento de funciones



Si $g(x)$ o algún $c^*g(x)$ es una cota superior de $f(x)$, para los $x > k$, se dice que

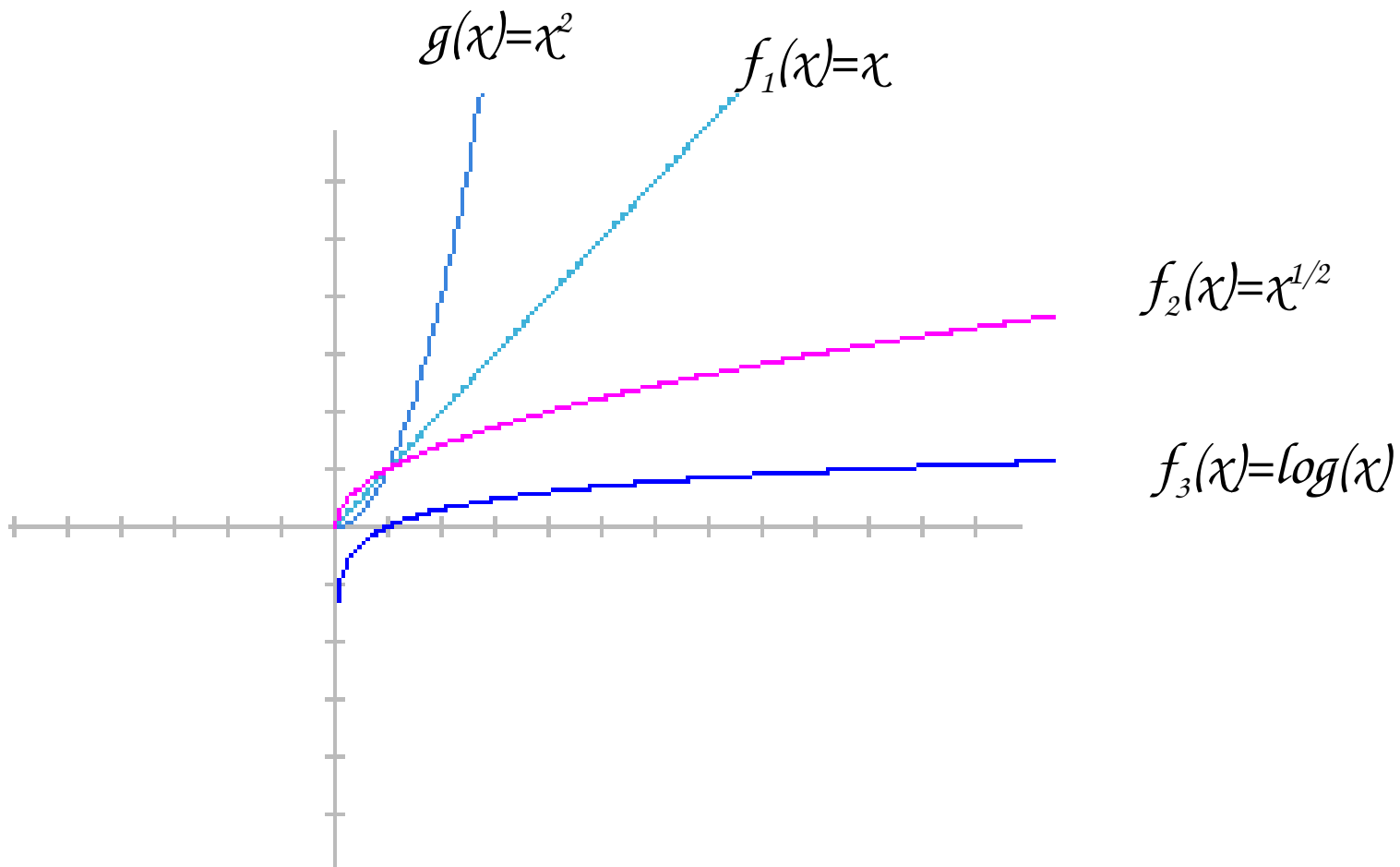
$f(x)$ es $O(g(x))$

Crecimiento de funciones

Dadas dos funciones f y g , se dice que $f(x)$ es $O(g(x))$ si existen constantes c y k tales que:

$$f(x) \leq c * g(x)$$

se cumple para todos los $x \geq k$



$g(x)$ es una cota superior de $f_1(x)$, $f_2(x)$, $f_3(x)$

Crecimiento de funciones

Formalmente la notación O representa un conjunto de funciones

*$O(g(x)) = \{f(x) \mid \text{existen constantes positivas } c \text{ y } k, \text{ tales que } 0 \leq f(x) \leq c$
* $g(x), \text{ para todos los } x \geq k\}$*

Crecimiento de funciones

$$O(x^2) = \{x, x^{1/2}, \log(x), \dots\}$$

Crecimiento de funciones

Muestre que $7x^2 = O(x^3)$

Crecimiento de funciones

Muestre que $7x^2 = O(x^3)$

$$7x^2 < x^3 \text{ para } x > 7 \quad C=1$$

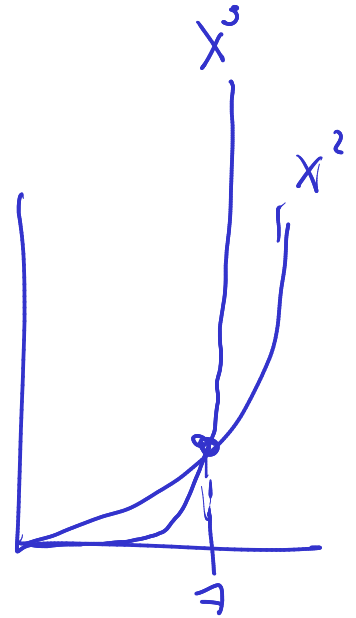
$$\begin{aligned} \exists x^2 &\leq C \cdot x^3 & x > K \\ \exists x^2 &< x^3 & x \neq 0 \\ \exists x &> 7 & \boxed{x > 7} \end{aligned}$$

Por lo tanto se cumple que

$$f(x) \leq 1 g(x)$$

para $x > 7$

$$C=1, k=7$$



Crecimiento de funciones

Es x^3 , $O(7x^2)$

Crecimiento de funciones

Es x^3 , $O(7x^2)$

$$x^3 \leq c_1 7x^2 \quad (x > k)$$
$$(k, \infty)$$

$$x^3 < c_1 7x^2$$

$$x < 7c_1$$

Ya que no se cumple para todos los $x > k$, no es cierto que x^3 sea $O(7x^2)$

Crecimiento de funciones

Ejercicios

Demostrar que:

- $x^2 + 2x + 1$ es $O(x^2)$
- $\log n$ es $O(n)$, parta sabiendo que $n < 2^n$
- $x^2 + 4x + 17$ es $O(x^3)$ pero que x^3 no es $O(x^2 + 4x + 17)$
- $x \log x$ es $O(x^2)$

Explique qué significa que una función sea $\Omega(1)$

Explique qué significa que una función sea $O(1)$

$$X^2 + 2x + 1 \leq C \cdot X^2 \quad x > k \quad \exists p(x) \rightarrow c > 0$$

$$X^2 + 2x + 1 \leq 2X^2$$

$$\lim_{x \rightarrow \infty} X^2 + 2x + 1 \leq 2X^2$$

$$\lim_{x \rightarrow \infty} \frac{X^2}{X^2} + \frac{2x}{X^2} + \frac{1}{X^2} \leq 2 \frac{X^2}{X^2}$$

$$1 \leq 2$$

$$X^2 + 2x + 1 \leq 2X^2$$

$$0 \leq X^2 - 2x - 1$$

$$a=1 \quad b=-2 \quad c=-1$$

$$ax^2 + bx + c = 0$$

$$\frac{-b \pm \sqrt{b^2 - 4ac}}{2a}$$

$$\frac{2 \pm \sqrt{4 - 4(-1)}}{2}$$

$$8 = 4 \times 2$$

$$\frac{2 \pm \sqrt{8}}{2}$$

$$= \frac{2 \pm 2\sqrt{2}}{2}$$

$$1 \pm \sqrt{2} \rightarrow 1 + \sqrt{2} \approx 2.42$$

$$X^2 + 2x + 1 \leq 2X^2 \quad x > 2.42$$

$$9 + 6 + 1 \leq 2 \times 9$$

$$16 \leq 18 \quad \checkmark$$

$$x^2 + 2x + 1 \leq c \cdot x^2$$

$$\lim_{x \rightarrow \infty} x^2 + (2x + 1) \leq c \cdot x^2$$

$$1 \leq c$$

$$c > 1$$

→ arbitrary

Estrategia

$$f(x) = O(g(x)), f(x) \leq c * g(x), \exists c > 0, \exists k > 0$$

La idea es encontrar un valor de c o k que cumplan la condición.

1. Puede establecer un valor de c y así hallar el k
2. Puede demostrar que la relación se cumple dando un valor a c y obteniendo el límite
3. Condicionar k a partir de c

Crecimiento de funciones

Cota superior asintótica

Cuando se determina que el tiempo de computo de un algoritmo es $O(g(x))$ se establece una cota superior

Dentro del análisis para obtener la cota superior, se debe considerar el peor caso, de esta forma se consigue tener una cota que no puede resultar peor

Crecimiento de funciones

Cota superior asintótica

Suponga que para el algoritmo 1 se encontró que $T_1(n)=O(n^2)$ y para el algoritmo 2 que $T_2(n)=O(n \lg n)$, qué representan estos resultados?

¿Qué se puede esperar en cuanto al tiempo de ejecución de los algoritmos?

¿Se puede asegurar que siempre es mejor el algoritmo 2 que el 1?

Crecimiento de funciones

Cota superior asintótica

Es un abuso decir que el tiempo de ejecución del insertion sort es $O(n^2)$, esto se puede asegurar, para el peor caso.

Crecimiento de funciones

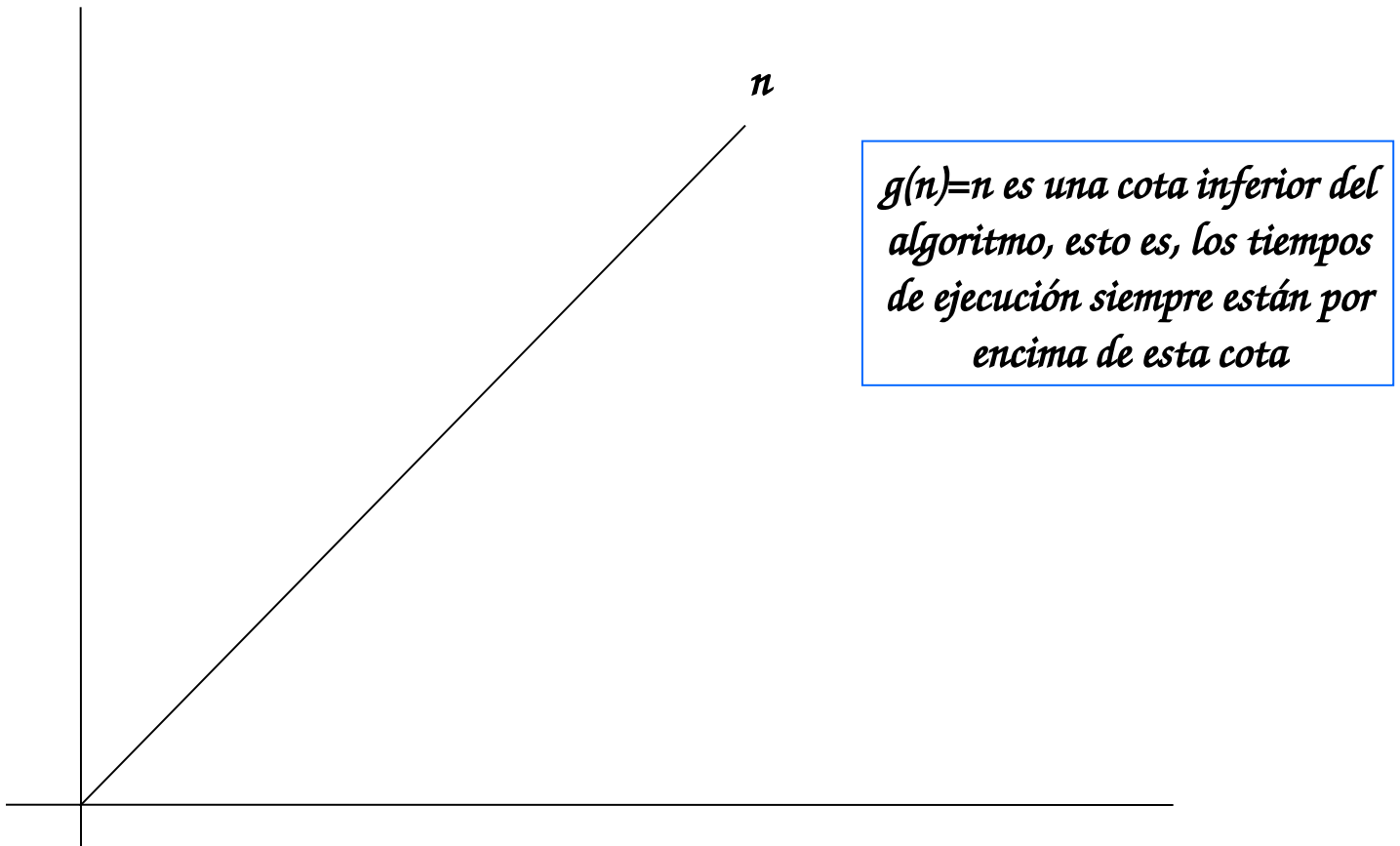
Cota inferior

$\Omega(g(x)) = \{f(x) \mid \text{existen constantes positivas } c \text{ y } k, \text{ tales que } 0 \leq c \cdot g(x) \leq f(x), \text{ para todos los } x \geq k\}$

Crecimiento de funciones

Cota inferior

Suponga que para un algoritmo se encontró que $T_1(n) = \Omega(n)$



Crecimiento de funciones

Cota inferior

Suponga que para un algoritmo se encontró que $T_1(n)=\Omega(n^2)$ y para otro que $T_2(n)=\Omega(n \lg n)$, qué representan estos resultados?

¿Qué se puede esperar en cuanto al tiempo de ejecución de los algoritmos?

Crecimiento de funciones

Notación $\Theta(f(n))$

$$f(n) = \Theta(g(n)) \text{ si } f(n) = O(g(n)) \text{ y } f(n) = \Omega(g(n))$$

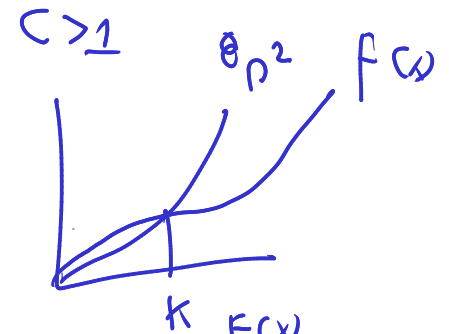
$$n^2 + 8n + 1 \sim \Theta(n^2)$$

1) $O(n^2)$

$$n^2 + 8n + 1 \leq C \times n^2$$

$$n^2 + 8n + 1 \leq 8n^2$$

$$7n^2 - 8n - 1 \geq 0 \quad \checkmark$$



$\Omega(n^2)$

$$n^2 + 8n + 1 \geq C \times n^2$$

$$C < 1$$

$$n^2 + 8n + 1 \geq \frac{1}{2} n^2$$



Crecimiento de funciones

Notación o

$o(g(x)) = \{f(x) \mid \text{existen constantes positivas } c \text{ y } k, \text{ tales que } 0 \leq f(x) < c \cdot g(x), \text{ para todos los } x > k\}$

Crecimiento de funciones

Notación o

$$2n^2 \text{ no es } o(n^2)$$

$$2n^2 < C \cdot n^2$$

La cota $2n^2 = O(n^2)$ es asintóticamente ajustada pero $2n = O(n^2)$ no.

Se utiliza la notación o para denotar cota superiores que no son asintóticamente ajustadas

$2n = o(n^2)$ pero $2n^2 = o(n^2)$ no.

$$o(n^2) = \{n \log n, n, 1, \dots\}$$

Crecimiento de funciones

Notación ω

$$\omega(n^2) = \{ n^3, n^4, n^2 \log(n), n! \} \rightarrow$$

$\omega(g(x)) = \{ f(x) \mid \text{existen constantes positivas } c \text{ y } k, \text{ tales que } 0 \leq c \cdot g(x) < f(x), \text{ para todos los } x \geq k \}$

Crecimiento de funciones

Complejidades de los algoritmos

<u>NS</u>	Complejidad	Terminología
P	$O(1)$	Complejidad <i>constante</i>
	$O(\log n)$	Complejidad <i>logarítmica</i>
	$O(n)$	Complejidad <i>lineal</i>
	$O(n \log n)$	Complejidad <i>$n \log n$</i>
	$O(n^b)$	Complejidad <i>polinomial</i>
NP	$O(b^n)$	Complejidad <i>exponencial</i>
	$O(n!)$	Complejidad <i>factorial</i>

Crecimiento de funciones

Complejidades de los algoritmos

Un problema que se puede resolver utilizando un algoritmo con complejidad polinómica en el peor caso se llama *tratable*. De no ser así se llama *intratable*

Un problema para el cual no existe un programa que lo pueda solucionar se llama *irresoluble*. De no ser así se llaman *resolubles*. (Halting problem – Turing)

Crecimiento de funciones

Complejidades de los algoritmos

Existen problemas que no se pueden resolver, en el peor caso en tiempo polinomial, pero que dada alguna solución, se puede comprobar que es efectivamente una solución. Estos problemas se llaman $\mathcal{NP}$ (Polinómico – No determinista)

Existe un conjunto de problemas $\mathcal{NP}$ que además, si se llegase a encontrar una solución en tiempo polinómico, daría solución a un conjunto de problemas relacionados. Este tipo de problema se conoce como $\mathcal{NP}$ -completos

Crecimiento de funciones

Complejidades de los algoritmos

El problema de la *satisfactibilidad* es un problema $\mathcal{NP}$ -completo.

Dada una asignación de valores de verdad, se puede verificar en tiempo polinómico si tal asignación satisface, o no, una fórmula proposicional. Sin embargo, no existe un algoritmo que en tiempo polinómico pueda encontrar la asignación de valores de verdad para que una fórmula cualquiera se satisfaga.

$$(x \vee y \vee z) \wedge (a \vee \bar{b} \vee x) \underline{\hspace{2cm}}$$

Crecimiento de funciones

Complejidades de los algoritmos

Además, si se encontrara un programa que lograra hallar la solución en tiempo polinómico, se podría solucionar un conjunto de problemas relacionados con la lógica proposicional